

A Survey of Testing Techniques Based on Large Language Models

Fei Qi*

Harbin Space Star Data System
Technology Co., Ltd, Harbin 150028,
China
qifei@spacestar.com.cn

Yingnan Hou

Harbin Space Star Data System
Technology Co., Ltd, Harbin 150028,
China
houyn@spacestar.com.cn

Ning Lin

Harbin Space Star Data System
Technology Co., Ltd, Harbin 150028,
China
linning@spacestar.com.cn

Shanshan Bao

Harbin Space Star Data System
Technology Co., Ltd, Harbin 150028,
China
baoss@spacestar.com.cn

Nuo Xu

Harbin Space Star Data System
Technology Co., Ltd, Harbin 150028,
China
xunuo@spacestar.com.cn

ABSTRACT

With the development of software testing technology, Large Language Model (LLM) driven testing method have gradually become an emerging trend in the field of software testing. This paper presents a comprehensive review of LLM-based testing techniques. The results of 19 studies using LLM to optimize testing techniques are analyzed from the perspective of software testing. This paper discusses in detail how to use LLM to optimize test techniques for generating automated test code and generating diverse input in software test tasks. It also summarizes the challenges and opportunities faced by this field. The above conclusions can identify the shortcomings of LLM-based software testing technology and the direction of future research.

CCS CONCEPTS

• **Computing methodologies** → Artificial intelligence; Natural language processing; Natural language generation.

KEYWORDS

LLM, Software Testing Techniques, Pre-trained Large Language Model

ACM Reference Format:

Fei Qi*, Yingnan Hou, Ning Lin, Shanshan Bao, and Nuo Xu. 2024. A Survey of Testing Techniques Based on Large Language Models. In *2024 International Conference on Computer and Multimedia Technology (ICCMT 2024)*, May 24–26, 2024, Sanming, China. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3675249.3675298>

1 INTRODUCTION

Software testing technology has made great progress in recent years, but there are still many challenges that have not been effectively solved. For example, the first challenge is to generate automated

test case code. Although there are many strategies, including hybrid search strategy, evolutionary algorithm search strategy, cycle and constraint strategy, and random strategy based on feedback information, the coverage and efficiency of generated test cases are still not satisfactory. The second challenge is to automate UI testing. Random rule-based techniques or mutation model-based techniques or reinforcement learning-based techniques are unable to deeply understand the semantic information of UI pages and often fail to achieve high coverage. Considering the limitations of the above studies, this motivates the exploration and application of new methods. Using LLM to upgrade software testing technology is the most potential way.

Large language models can be applied to various language tasks and can also be adapted to the task requirements of specific domains through fine-tuning techniques, including innovative applications such as code generation techniques and automated testing techniques. Although LLM has become a key support to promote the technology upgrading in the field of testing, the comprehensive and systematic research on the application of LLM is relatively lagging behind in all stages of testing. This type of research lag may limit the effective application of LLM in the testing process. Therefore, it becomes particularly important to strengthen the research and summary of the application of LLM in all stages of testing, which will help to promote the further development of LLM and testing technology. A total of 19 related literatures are surveyed in this paper. This paper focuses on how to generate automated test code efficiently, how to generate diverse test inputs. The purpose of this paper is to provide a guide for researchers in the field of testing to quickly understand the latest progress and the urgent challenges in this field.

2 A SURVEY OF LARGE LANGUAGE MODELS

After entering the 21st century, natural Language models can be roughly divided into Neural Networks Language Model and Large-scale language Model according to development stage. In 2003, Bengio et al. proposed the concept of language model based on neural network, which opened a new direction and idea of natural language processing. In 2015, Dai et al. first expounded the basic idea of large language models, which laid the foundation for the development of subsequent large language models. In 2017, Vaswani

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICCMT 2024, May 24–26, 2024, Sanming, China

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1826-7/24/05

<https://doi.org/10.1145/3675249.3675298>

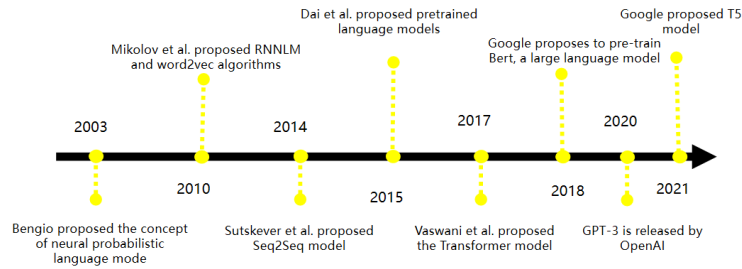


Figure 1: Language Model Development sequence

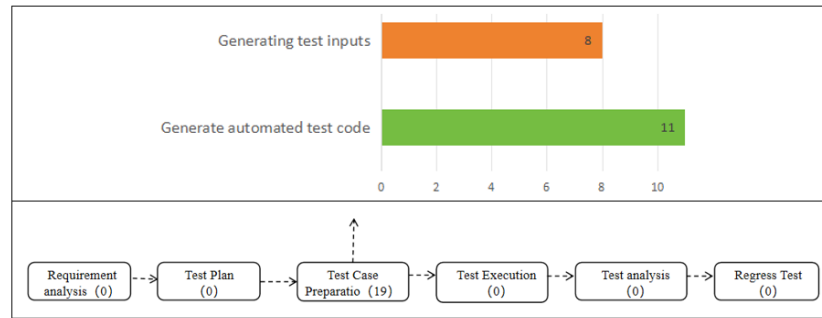


Figure 2: Distribution of testing cycle

et al. proposed a large language model Transformer based on attention mechanism in their paper, which has important application value in the field of natural language processing and computer vision. From 2018 to 2021, Google and OpenAI teams have successively launched Bert model, GPT-3 application, and T5 model, which have received extensive attention worldwide. The above text is shown in Figure 1.

3 TESTING TECHNOLOGY BASED ON LARGE LANGUAGE MODEL

Figure 2 lists the research priorities and the number of literature involved in different phases of the test cycle. As shown in the figure, this section focuses on the analysis of testing techniques in the test preparation phase, which includes generating automated test code and generating diverse inputs.

3.1 Generate automated test code

The ability of LLM to understand source code and produce complex code has been widely recognized. Due to the difference between general code generation and test case code generation, it shows that LLM needs to adopt pre-training or fine-tuning mode for optimization.

Pre-train or fine-tune to generate automated test code. Alagarsamy et al. [1] design a self-supervised pre-training mode, so that the PLBART architecture uses the bidirectional attention mechanism to capture the context information of the assertion corpus, and then follows the domain adaptation principle to fine-tune the generated test cases, aiming to build the relationship between the

attention mechanism and the test cases. Compared with the AthenaTest tool in Defects4j dataset, the accuracy of test cases is improved by 147%, and the coverage rate is improved by 15%. Hashtroudi et al. [2] proposed to use CodeT5 to learn general syntactic, semantic and structural information, and used the domain-specific dataset Methods2test to fine-tune CodeT5. Compared with CodeT5 without fine-tuning in Defects4j dataset, the line coverage of test cases is improved by 18.62%. Rao et al. [3] proposed CAT-LM model, which can understand code semantics, test logic and assertions at the same time. The model can learn the mapping relationship and pattern between code and test data, increased Max input to 8192 tokens, so as to generate automated test cases that are more in line with test requirements than the Teco model. Steenhoek et al. [4] used anti-patterns to guide LLM to learn and adjust the generation strategy of test cases, and the feedback signals were diversified static quality indicators, such as test execution results, code coverage, etc. In the C# open source dataset, the optimized model increases 23.1% valid assertions and reduces 1.8% repeated assertions and 2.4% abnormal test smell compared with GPT-4 without fine-tuning.

Design prompt word to generation automatic test code. Dakhel et al. [5] proposed to use mutation testing technology to enhance the sensitivity and recognition ability of LLM to erroneous codes. Because mutants can indicate vulnerabilities in codes, the trained model can provide test cases that can detect one or more mutants. Xie et al. [6] designed an adaptive focus context hint word, and merged the preset dependency information of the hint word into the feature information to generate test cases. In the test of the code generated by LLM Codex, the optimized model successfully identified 86.72% of the defects. Li et al. [7] used the code interpreter to

Table 1: Performance of automated testing

	Dataset	Metrics	GPT-3.5-Turbo	StarCoder	Codex(2K)
Generate test code accuracy	HumanEval	correct	52.3%	81.3%	77.5%
		partially correct	92.3%	81.3%	87.5%
	SF110	correct	6.9%	51.9%	46.5%
		partially correct	16.1%	58.6%	62.7%
Generate test code coverage	HumanEval	line coverage	69.1%	67.0%	87.4%
		branch coverage	76.5%	69.3%	92.1%
	SF110	line coverage	1.3%	1.1%	1.9%
		branch coverage	1.6%	0.5%	1.1%

guide the LLM to understand the internal structure characteristics of the code, and designed QuixBugs error set to prompt the LLM to pay attention to typical error patterns, which provided effective support for the code interpreter. Yuan et al. [8] proposed an automated test code generation method based on improved ChatGPT. Firstly, a code initial generator and an iterative test refiner are designed, and the iterative test refiner follows the verify-fix paradigm. Secondly, the iterative test refiner fixes the error messages generated by the initial generator in the test, which prompts the LLM based on the error message and the focus code context. It is worth noting that although the model structure has a certain impact on the function of the model, the choice of training target and dataset is a key factor in determining the end use of the model. After in-depth experimental demonstration [9], the performance of three tools, Codex, StarCoder and GPT-3.5-Turbo, is compared in test case generation. The experimental results are shown in Table 1. CodeX achieves the best performance in test case code generation. However, we can see that in the SF110 benchmark, none of the three tools evaluated had more than 2% coverage.

Design additional files to generate automated test code. Vikram et al. [10] explored the ability of LLM to generate automated test cases with the aid of system property documentation. The authors believe that system properties are closely related to API documentation, which involves functions, input parameters, output results, and possible error conditions. Documentation can provide accurate information to LLM. The theory can generate test cases that meet the required properties. Plein et al. [11] summarized bugs into problem reports, and the LLM used the problem and exception information recorded in the problem reports to generate test cases for specific problems. The experimental results show that although only 6% of the generated test cases are valid, the related theory is proved to be feasible.

3.2 Generating test inputs

We present the generated test data from two different perspectives in detail: software types and testing techniques, which is helpful for comprehending and evaluating related research results comprehensively and systematically.

3.2.1 Test inputs are generated based on the software type. Generate test inputs for GUI application testing. Behavioral diversity of test inputs is a prominent problem that hinders test coverage. Liu [12] et al. proposed InputBlaster that is a model based on mutation rule and

LLM. LLM is used to automatically generate initial input data, and then a batch of abnormal text inputs is generated under the guidance of mutation rules. Specifically, LLM first generates the initial input by GUI information and related context information. Then LLM generates mutation rules based on GUI feedback information and candidate constraints. Finally, the model updates the initial input according to the mutation rules to produce text information for anomaly detection. The experimental results show that the model can identify one defect on average 13.52 times, and the average detection time is 9.64 minutes. Liu et al. [13] proposed to use LLM to perceive GUI context semantic information, business logic and interaction history, and then automatically generate text input data in line with usage scenarios.

In addition to text as input, GUI can also be simulated in the form of virtual simulation. Liu et al. [14] proposed the GPTDroid, which extracts GUI page information. Then there are multiple question-and-answer interactions with the mobile app to deepen understanding of the page's information. Finally, the test script is generated and executed. In this framework, a function-aware memory cue mechanism is also introduced to guide exploration. In addition to the method of generating automated test code by LLM, Yu et al. [15] also discussed the method of using LLM for identification and iterative testing by simulating the pattern of human-computer interaction in the case of software upgrade. This method is applicable scenarios include identifying API changes, update interface elements, and changed software installation environment.

Generate test inputs for deep learning libraries testing. Test inputs need to satisfy the constraints of diverse tensor data structures. However, traditional API-level fuzzing techniques or frame-level fuzzing techniques have limitations. In order to solve the above problems, Deng et al. [16] proposed TitanFuzz framework based on fuzzing. In detail, the original program is first generated using a generative LLM (e.g., Codex) for fuzzing, then part of the code is replaced with a mask by using a mutation operator. The original code is updated by using LLM (e.g., InCoder) to generate new code replacing the mask. After further research [17], others subsequently designed FuzzGPT, which automatically generates data for edge cases or abnormal behaviors through the inherent capabilities of LLM (including fine-tuning and contextual learning). Experiments on deep learning libraries (PyTorch and TensorFlow) show that FuzzGPT outperforms TitanFuzz in terms of defect detection and coverage. It discovers 49 new bugs, including 11 high-priority bugs.

3.2.2 Test inputs are generated based on testing techniques. Generate test inputs by LLM fuzzing framework. Fuzzing is a widely used traditional testing technique, which mainly generates unexpected, invalid and random input data to evaluate the quality of software. LLM plays an important role in improving fuzzing technology and improving the generation of diverse inputs. Yao et al. [18] proposed FuzzLLM that is a fuzzy testing framework. The key idea of the design is that FuzzLLM makes use of templates to capture the structural integrity of prompts and isolate key characteristics of jailbreak classes as constraints. Diverse inputs are formed by integrating different base categories and varying constraints. Hu et al. [19] proposed ChatFuzz, an enhanced gray-box fuzzer based on LLM. Specifically, a seed is selected in the seed pool of the fuzzer. The ChatFuzz is prompted to make changes that are more consistent in line with format requirements. They experimented the coverage of different gray-box fuzzers to generate three types of inputs, and the edge coverage was improved by 12.77% compared with SOTA gray-box fuzzer (AFL++).

Generate test inputs by other testing techniques. Some scholars have used LLM to enhance GUI testing techniques for interactive exploration and context-aware generation of automated test scripts, which has been introduced in Section 3.2.1.

4 CHALLENGES AND OPPORTUNITIES OF LLM IN TESTING

4.1 Challenges

As the paper survey shows, research on the use of LLM to optimize software testing techniques has experienced significant growth in the past years. However, this technology is in its stage of development, and many challenges need to be solved.

In order to achieve high test coverage, exploring different behaviors of the software under test is an important direction in the field of testing. But it is still challenging for LLMS to directly achieve outputs with diversity. For example, according to the information provided in Table 1 of Section 3.1, the performance of large language models in generating test cases in SF110 dataset is much lower than using Evosuite [9]. Evosuite tool achieves 27.5% and 20.2% line coverage and branch coverage. However, the line coverage and branch coverage achieved by large language models do not exceed 2%.

As shown in Section 3.1, a high-quality dataset is an important factor affecting model performance. However, manually constructing high-quality datasets is time-consuming and labor-intensive. To solve this problem, research focuses on the use of specific tools or scripts to automatically collect and extract key information as a solution.

4.2 Opportunities

Figure 2 collates the application distribution of LLM in the testing cycle. As shown in the figure, LLM has not been applied in any phase except the mid-test phase. This paper encourages researchers to abstract domain-specific knowledge bases into machine language, using such knowledge to train LLM and apply it to the early stage, i.e. the test planning stage. Because this work requires specific

domain knowledge, it is very suitable for applying LLM to improve technology. But this research has not yet been explored.

Reason for not applying to the stages that is regression test: Probably due to insufficient diversity of inputs data. To solve this problem, we can refer to the design ideas in Sections 3.1 and 3.2, use LLM to learn prior knowledge or use LLM-based mutation techniques to generate diverse inputs.

5 CONCLUSION

This paper reviews the application of LLM in software testing, analyzes related research from the perspective of LLM-based testing technology, and summarizes the challenges and opportunities in this area. The results show that LLM has been successfully applied to test tasks by using pre-training and fine-tuning, including generating automated test code, generating test inputs. However, there are still challenges in achieving high test coverage, constructing datasets, and so on, which may become another research direction.

The above conclusions can help to understand the gap of LLM-based software testing technology, and serve as the direction of future research. We believe that the information provided in this paper can help relevant researchers to better apply LLM technology to improve testing technology, and ultimately improve the quality of software systems.

REFERENCES

- [1] Alagarsamy S, Tantithamthavorn C, Aleti A. A3test: Assertion-augmented automated test case generation[J]. arXiv preprint arXiv:2302.10352, 2023.
- [2] Hashtroudi S, Shin J, Hemmati H, et al. Automated test case generation using code models and domain adaptation[J]. arXiv preprint arXiv:2308.08033, 2023.
- [3] Rao N, Jain K, Alon U, et al. CAT-LM training language models on aligned code and tests[C]//2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE). IEEE, 2023: 409-420.
- [4] Steenhoek B, Tufano M, Sundaresan N, et al. Reinforcement Learning from Automatic Feedback for High-Quality Unit Test Generation[J]. arXiv preprint arXiv:2310.02368, 2023.
- [5] Dakhel A M, Nikanjam A, Majdinasab V, et al. Effective test generation using pre-trained large language models and mutation testing[J]. Information and Software Technology, 2024: 107468.
- [6] Xie Z, Chen Y, Zhi C, et al. ChatUniTest: a ChatGPT-based automated unit test generation tool[J]. arXiv preprint arXiv:2305.04764, 2023.
- [7] Li V, Doiron N. Prompting code interpreter to write better unit tests on quixbugs functions[J]. arXiv preprint arXiv:2310.00483, 2023.
- [8] Z. Yuan, Y. Lou, M. Liu, S. Ding, K. Wang, Y. Chen, and X. Peng, "No more manual tests? evaluating and improving chatgpt for unit test generation," arXiv preprint arXiv:2305.04207, 2023.
- [9] Siddiq M L, Santos J, Tanvir R H, et al. Exploring the effectiveness of large language models in generating unit tests [J]. arXiv preprint arXiv:2305.00418, 2023.
- [10] Vikram V, Lemieux C, Padhye R. Can large language models write good property-based tests?[J]. arXiv preprint arXiv:2307.04346, 2023.
- [11] Plein L, Ouédraogo W C, Klein J, et al. Automatic generation of test cases based on bug reports: a feasibility study with large language models[J]. arXiv preprint arXiv:2310.06320, 2023.
- [12] Liu Z, Chen C, Wang J, et al. Testing the limits: Unusual text inputs generation for mobile app crash detection with large language model[C]//Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. 2024: 1-12.
- [13] Liu Z, Chen C, Wang J, et al. Fill in the blank: Context-aware automated text input generation for mobile gui testing[C]//2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). IEEE, 2023: 1355-1367.
- [14] Liu Z, Chen C, Wang J, et al. Make LLM a testing expert: Bringing human-like interaction to mobile GUI testing via functionality-aware decisions[C]//Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. 2024: 1-13.
- [15] Yu S, Fang C, Ling Y, et al. LLM for test script generation and migration: Challenges, capabilities, and opportunities[C]//2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security (QRS). IEEE, 2023: 206-217.
- [16] Deng Y, Xia C S, Peng H, et al. Fuzzing deep-learning libraries via large language models[C]//Proceedings of the 32nd ACM SIGSOFT international symposium on

- software testing and analysis. 2023: 423-435.
- [17] Deng Y, Xia C S, Yang C, *et al*. Large language models are edge-case generators: Crafting unusual programs for fuzzing deep learning libraries[C]//Proceedings of the 46th IEEE/ACM International Conference on Software Engineering. 2024: 1-13.
- [18] Yao D, Zhang J, Harris I G, *et al*. Fuzzllm: A novel and universal fuzzing framework for proactively discovering jailbreak vulnerabilities in large language models[C]//ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). IEEE, 2024: 4485-4489.
- [19] Hu J, Zhang Q, Yin H. Augmenting greybox fuzzing with generative ai[J]. arXiv preprint arXiv:2306.06782, 2023.